AI-Programming Project 1

Jørgen Hauan Larsen

February 2026

1 Introduction

In this project I have programmed differentiable control of multiple dynamical systems using both a classical PID controller and a neural network, implemented with JAX. The three dynamical systems are a bathtub, Cournot competition and a leaky integrate-and-fire neuron, which are all regulated using the two controllers.

2 Results

2.1 Bathtub

2.1.1 PID

The PID controller successfully regulates the bathtub system to the desired water level. The learning curve shows fast and stable convergence with low variance across epochs. Due to the natural damping of the system, the control task is relatively easy, and the PID controller performs robustly even in the presence of noise. This run serves as a baseline for comparison with the neural network controller.

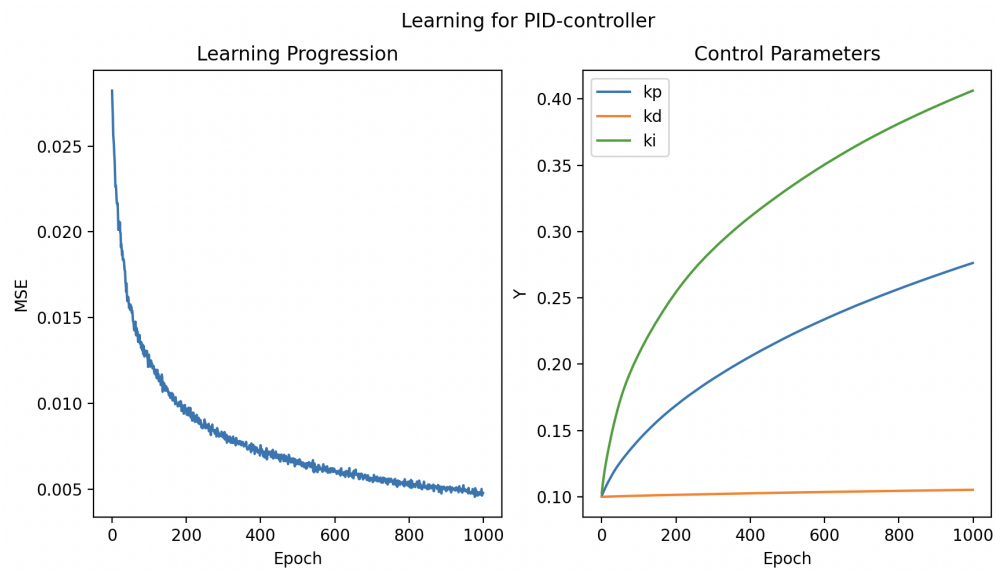

Figure 1: Learning curve and control parameters for PID control of the bathtub system

Table 1: Configuration used for the bathtub–PID experiment

Category	**Value**
Plant type	Bathtub
Controller type	PID
H_0	1.0
A	10.0
C	0.10
G	9.8
Target T	1.0
k_p	0.1
k_i	0.1
k_d	0.1
Epochs	1000
Timesteps	100
Learning rate	0.01
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

2.1.2 NN

Figure 2 shows the learning progression of the neural network controller applied to the bathtub system. The loss decreases smoothly and monotonically over the

training epochs, indicating stable learning and effective regulation of the water level toward the target.
During experimentation, it was observed that initializing the network parameters with a symmetric range around zero, or with a very small initialization range such as (0.01,0.01), often resulted in NaN values during training. To avoid this, the initialization range was restricted to non-negative values and increased to (0.0, 0.1). This choice leads to larger initial control signals but improves numerical stability during backpropagation through time. With this configuration, training remains stable over long horizons and converges reliably to a low tracking error.

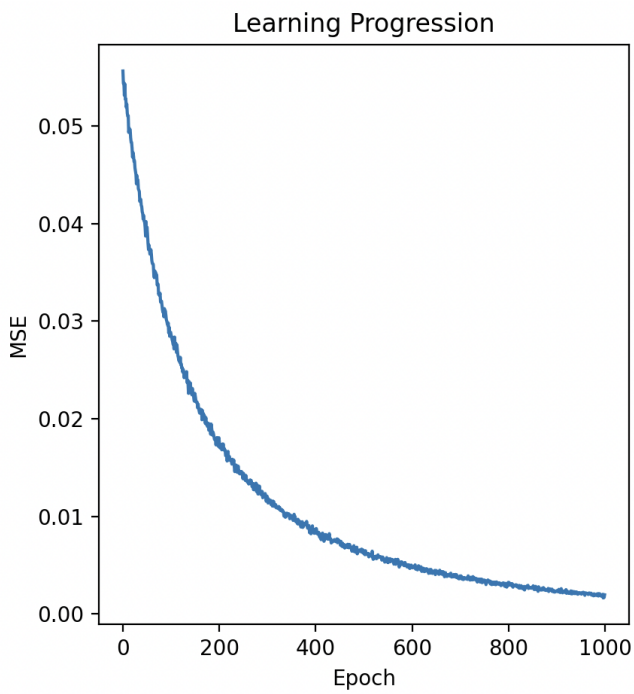

Figure 2: Learning curve for neural network control of the bathtub system

Table 2: Configuration used for the Bathtub–NN experiment

Plant type	Bathtub
Controller type	NN
H_0	1.0
A	10.0
C	0.10
G	9.8
Target T	1.0
Layers	[3, 16, 16, 16, 1]
Activation	ReLU
Init range	$(0.0, 0.1)$
Epochs	1000
Timesteps	100
Learning rate	0.001
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

2.2 Cournot

2.2.1 PID

The PID controller is able to reduce the tracking error for the Cournot competition model, as shown by the decrease in mean squared error during training (Figure 3). The learning curve exhibits a rapid initial drop followed by convergence to a stable plateau, indicating that the controller parameters settle into a locally optimal configuration. The evolution of the control parameters shows that the proportional gain k_p increases steadily, while the derivative and integral gains decay toward zero. This suggests that the learned control strategy relies primarily on proportional feedback, which is sufficient for stabilizing the constrained Cournot dynamics. Compared to the bathtub system, convergence is slower and the final error is higher, reflecting the increased complexity and nonlinearity of the economic model.

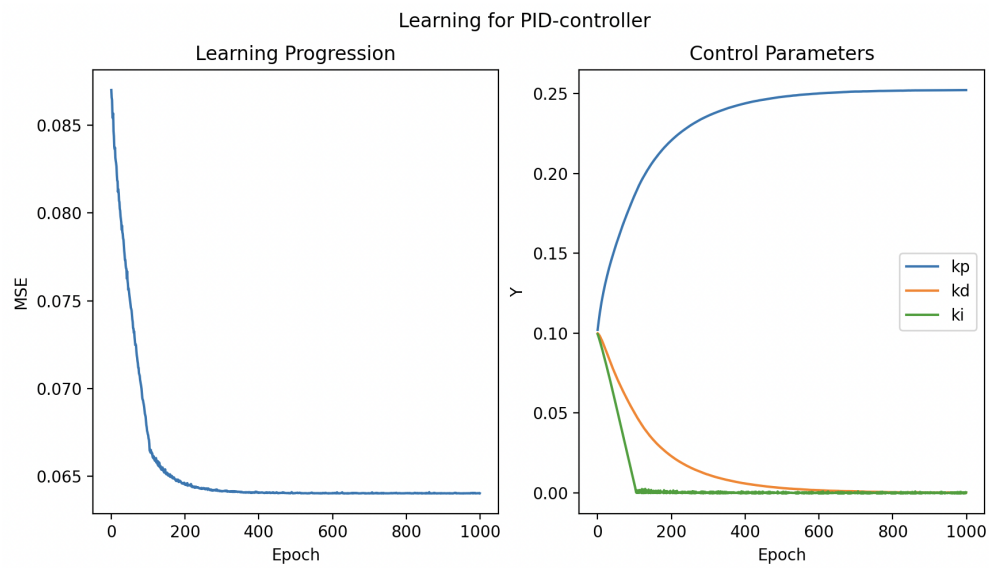

Figure 3: Learning curve and control parameters for PID control of the Cournot competition model

Table 3: Configuration used for the Cournot–PID experiment

Plant type	Cournot
Controller type	PID
$p_{\max}$	5.0
c_m	0.1
Initial q	0.1
Target T	3.0
k_p	0.1
k_i	0.1
k_d	0.1
Epochs	1000
Timesteps	100
Learning rate	0.01
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

2.2.2 NN

Figure 4 shows the learning progression of the neural network controller applied to the Cournot competition model. I had to remove hard clipping of the production quantities. Instead of explicitly bounding the quantities to the interval [0,1], a sigmoid function was used to ensure that the outputs remained within

valid bounds. Under this formulation, the loss decreases smoothly over training and converges toward a stable value. Compared to the results I got with clipped quantities, the learning behavior is more stable and does not exhibit large spikes or numerical divergence.

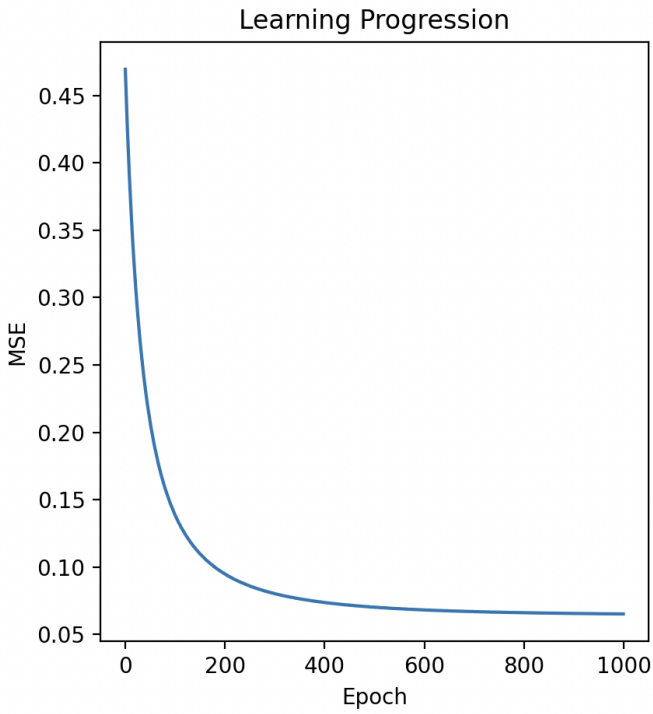

Figure 4: Learning curve for neural network control of the Cournot model

Table 4: Configuration used for the Cournot–NN experiment

Plant type	Cournot
Controller type	NN
$p_{\max}$	5.0
c_m	0.1
Initial q	0.1
Target T	3.0
Layers	[3, 16, 16, 16, 1]
Activation	ReLU
Init range	$(-0.01, 0.01)$
Epochs	1000
Timesteps	100
Learning rate	0.01
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

2.3 Leaky Integrate-and-Fire Neuron

2.3.1 PID

Figure 5 shows the learning progression of the PID controller applied to the leaky integrate-and-fire (LIF) neuron model. The loss decreases steadily during the early training epochs and then converges toward a stable plateau, indicating consistent improvement in tracking performance. The evolution of the control parameters shows that the proportional and derivative gains increase gradually over training, while the integral gain decreases rapidly and remains close to zero. This suggests that proportional and derivative feedback play a more prominent role in regulating the firing rate, whereas integral action contributes less in this setting. Overall, the learning process remains stable, although convergence is slower compared to simpler plant models.

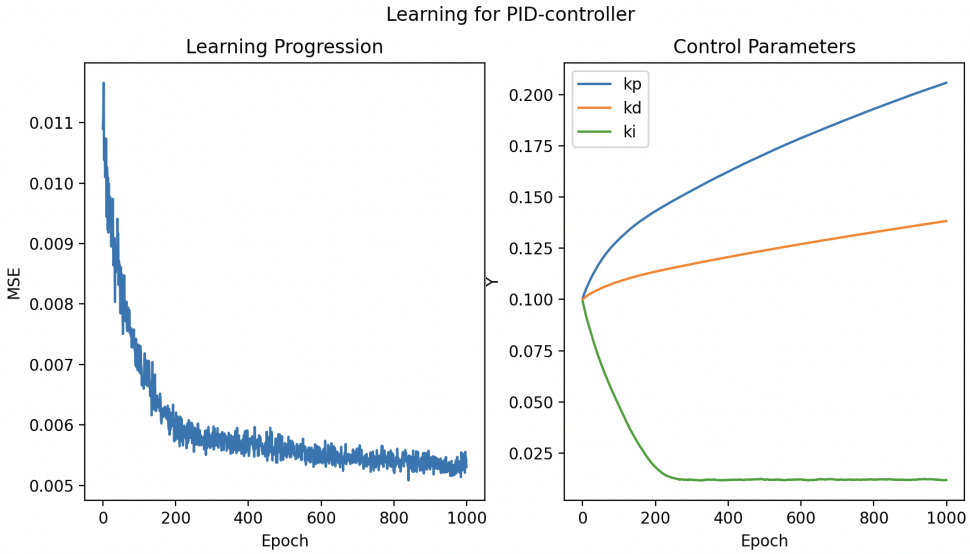

Figure 5: Learning curve and control parameters for PID control of the leaky integrate-and-fire neuron

Table 5: Configuration used for the LIF–PID experiment

Plant type	LIF
Controller type	PID
Spike threshold	-0.05
V_0	-0.07
α	0.7
γ	0.2
Target T	0.5
k_p	0.1
k_i	0.1
k_d	0.1
Epochs	1000
Timesteps	100
Learning rate	0.01
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

2.3.2 NN

Figure 6 shows the learning progression of the neural network controller applied to the leaky integrate-and-fire neuron model. The loss decreases rapidly during the initial training phase, indicating early improvement in tracking the target firing rate. After this initial reduction, the loss converges toward a stable plateau

with moderate fluctuations over the remaining epochs. In this run, a smaller learning rate was used compared to earlier experiments, which contributes to stable training. Additionally, the neural network architecture was reduced to two hidden layers with two neurons each, limiting model complexity while still allowing the controller to capture the essential dynamics of the system. Overall, the neural network achieves consistent regulation of the firing rate, albeit with residual variability due to stochastic disturbances and threshold-based spiking behavior. Using a larger network (3, 16, 16, 16, 1) resulted in smaller variance, but with loss converging between 0.005 and

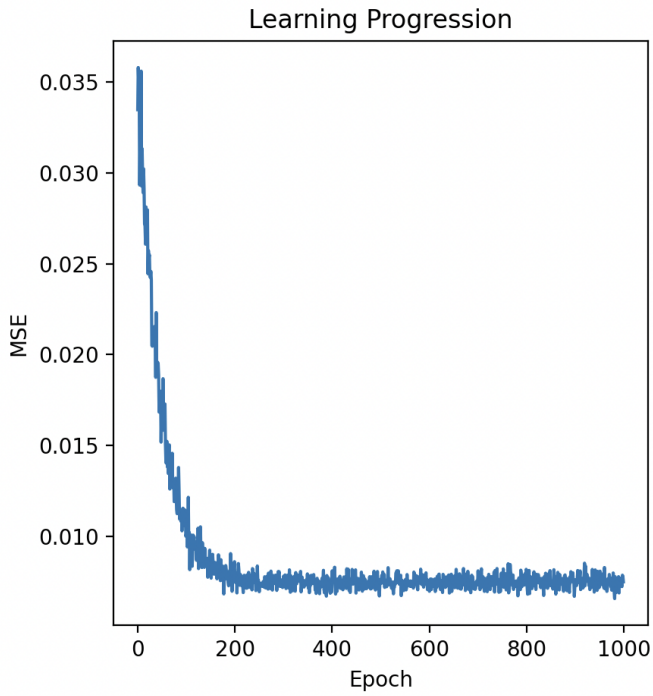

Figure 6: Learning curve for neural network control of the LIF neuron

Table 6: Configuration used for the LIF–NN experiment

Plant type	LIF
Controller type	NN
Spike threshold	-0.05
V_0	-0.07
α	0.7
γ	0.2
Target T	0.5
Layers	[3, 16, 16, 16, 1]
Activation	ReLU
Init range	$(-0.01, 0.01)$
Epochs	1000
Timesteps	100
Learning rate	0.0001
Seed	42
Noise D	$\{-0.01, -0.005, 0, 0.005, 0.01\}$

3 Leaky Integrate-and-Fire Neuron Model

The Leaky Integrate-and-Fire Neuron models a neuron getting input from several other neurons and firing whenever the membrane potential becomes higher than a certain threshold. The controller is one of the input neurons and the noise model external neurons. Average firing rate of the neuron is the target for the controller.

The state of the system at time step t is defined as

$$x_t = (V_t, r_t),$$

where V_t denotes the membrane potential and r_t denotes an averaged firing rate.

The membrane potential evolves according to the discrete-time update equation

$$V_{t+1} = \alpha V_t + U_t + D_t,$$

where $\alpha \in (0, 1)$ is the leak parameter, U_t is the control input, and D_t is an external disturbance.

A spike is generated whenever the membrane potential exceeds a fixed threshold V_{thr}:

$$s_t = \{\, 1 \,, V_t \geq V_{thr}, 0, otherwise.$$

If a spike occurs, the membrane potential is reset to a baseline value V_0:

$$V_{t+1} = V_0 \quad if s_t = 1.$$

Rather than using the binary spike signal directly as output, a smoothed firing rate is computed:

$$r_{t+1} = (1 - \gamma) r_t + \gamma s_t,$$

where $\gamma \in (0, 1)$ controls the averaging window.
The firing rate r_t is used as the plant output:

$$y_t = r_t.$$

The LIF model combines linear membrane dynamics with a threshold-based spiking mechanism and a reset rule, resulting in a hybrid system with both continuous and discrete components. While the membrane potential dynamics are linear, the spike generation introduces non-smooth behavior, which can make control and learning more challenging compared to the other plant models considered in this project.